

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



EVIDENCIAS PRUEBAS FUNCIONALES QUERIES



Ezequiel Pino

Pruebas Funcionales Queries

Objetivo	2
Q1 — Aplicaciones con categoría ORO	2
División de Q1 en dos consultas	2
Prueba funcional Q1-C — Conteo	3
Prueba funcional Q1-L — Listado	3
Prueba de navegación desde los resultados	4
Resultado de Q1	5
Q2 — Aplicaciones por estado de Vigencia	6
Q3 — Impacto por baja de Base de Datos 28	7
Identificación de Base de Datos 28	7
División de Q3 en dos consultas	7
Prueba funcional Q3-C — Cantidad de aplicaciones afectadas	8
Prueba funcional Q3-L — Listado de aplicaciones afectadas	9
Exclusión de elementos no aplicación	10
Prueba de navegación desde Q3-L	10
Verificación de la relación con Base de Datos 28	11
Resultado final de Q3	12

Pruebas Funcionales Queries

Objetivo

El objetivo de estas pruebas es validar en Enterprise Architect, las consultas SQL desarrolladas para responder los interrogantes de gobierno planteados en el Ejercicio 2.

Las consultas fueron previamente verificadas mediante ejecución read-only sobre el repositorio .gea utilizando SQLite. Sin embargo, la validación funcional definitiva se realiza en el motor de búsqueda SQL de Enterprise Architect, obviamente, comprobando tanto los resultados obtenidos como su correspondencia con los elementos visibles del modelo. Vamos a testear una a una.

Q1 – Aplicaciones con categoría ORO

La primera pregunta planteada por el desafío es:

¿Cuántos elementos de tipo aplicación tienen el tagged value Categoría con valor ORO? ¿Cuáles son?

Para responder, se utilizaron las tablas `t_object` y `t_objectproperties`.

La aplicación se identifica mediante:

```
Object_Type = 'Class'  
Stereotype = 'ArchiMate_ApplicationComponent'
```

Mientras que el valor de categoría se obtiene desde `t_objectproperties`, relacionándolo con el elemento mediante `Object_ID` y filtrando:

```
Property = 'Categoría'  
Value = 'ORO'
```

División de Q1 en dos consultas

Aunque Q1 representa una única pregunta de negocio, se decidió dividirla en dos consultas SQL independientes:

- **Q1-C — Categoría ORO — Count:** obtiene exclusivamente la cantidad total de aplicaciones que cumplen la condición.
- **Q1-L — Categoría ORO — List:** devuelve el detalle de las aplicaciones encontradas.

Esta separación permite mantener resultados homogéneos y más fáciles de interpretar en Enterprise Architect. La consulta de conteo devuelve una única fila agregada, mientras que la consulta de listado devuelve elementos individuales del modelo.

Además, en Q1-L se incorporaron las columnas especiales **CLASSGUID** y **CLASSTYPE**, utilizadas por Enterprise Architect para asociar cada fila del resultado con su elemento correspondiente y permitir la navegación directa desde los resultados de búsqueda.

De esta forma, el conteo y la trazabilidad de los elementos se resuelven de manera independiente sin mezclar información agregada con filas navegables.

Prueba funcional Q1-C – Conteo

Se creó en Enterprise Architect una búsqueda SQL denominada:

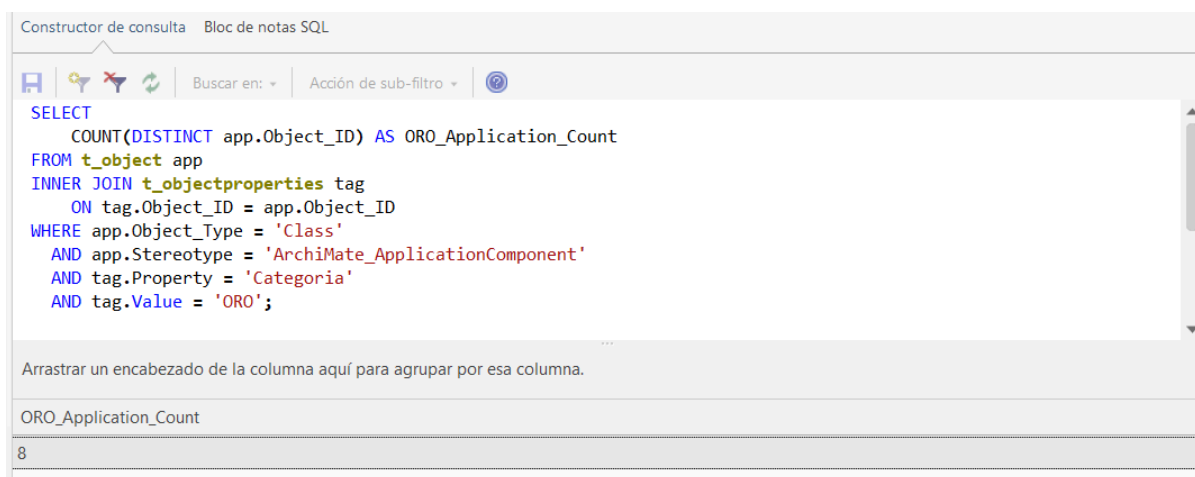
Q1-C — Categoría ORO — Count

La consulta fue ejecutada desde el editor SQL de la función **Buscar en el proyecto**.

El resultado obtenido fue:

ORO_Application_Count = 8

El valor coincide con el resultado previamente obtenido durante la validación diagnóstica read-only del repositorio.



Prueba funcional Q1-L – Listado

Se creó una segunda búsqueda SQL denominada:

Q1-L — Categoría ORO — List

La consulta devolvió ocho aplicaciones:

Aplicación 1, Aplicación 2, Aplicación 20, Aplicación 29, Aplicación 3, Aplicación 4, Aplicación 6, Aplicación 8

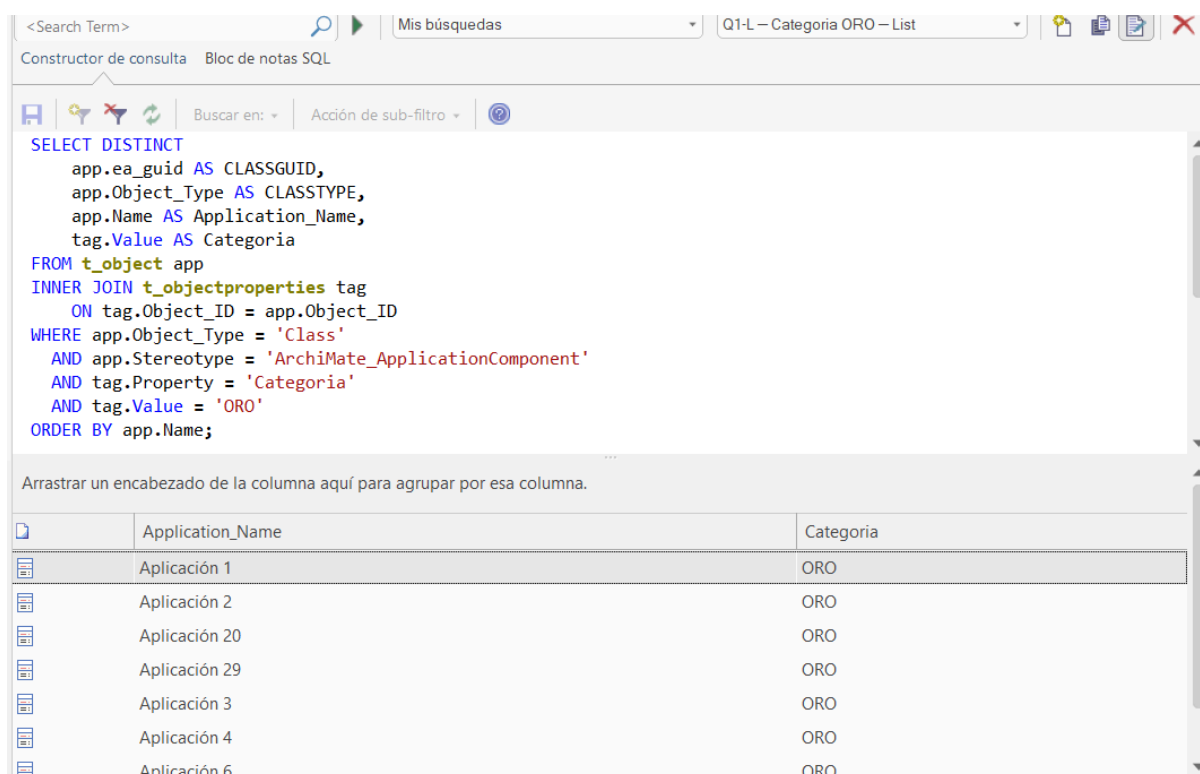
El orden observado corresponde al orden alfabético aplicado sobre el campo **Name**; por lo tanto, nombres como **Aplicación 20** aparecen antes de **Aplicación 3**.

El conjunto de aplicaciones obtenido coincide exactamente con el resultado esperado:

Aplicación 1, Aplicación 2, Aplicación 3, Aplicación 4, Aplicación 6, Aplicación 8, Aplicación 20, Aplicación 29

Todas las filas mostraron además:

Categoria = ORO



The screenshot shows the SQL Editor window with the following query:

```
SELECT DISTINCT
    app.ea_guid AS CLASSGUID,
    app.Object_Type AS CLASSTYPE,
    app.Name AS Application_Name,
    tag.Value AS Categoria
FROM t_object app
INNER JOIN t_objectproperties tag
    ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
    AND app.Stereotype = 'Archimate_ApplicationComponent'
    AND tag.Property = 'Categoria'
    AND tag.Value = 'ORO'
ORDER BY app.Name;
```

Below the query, the results are displayed in a table with the following columns: Application_Name and Categoria. The results are sorted alphabetically by Application_Name.

Application_Name	Categoria
Aplicación 1	ORO
Aplicación 2	ORO
Aplicación 20	ORO
Aplicación 29	ORO
Aplicación 3	ORO
Aplicación 4	ORO
Aplicación 6	ORO

No se observaron elementos que no fueran aplicaciones.

Prueba de navegación desde los resultados

La consulta Q1-L incorpora:

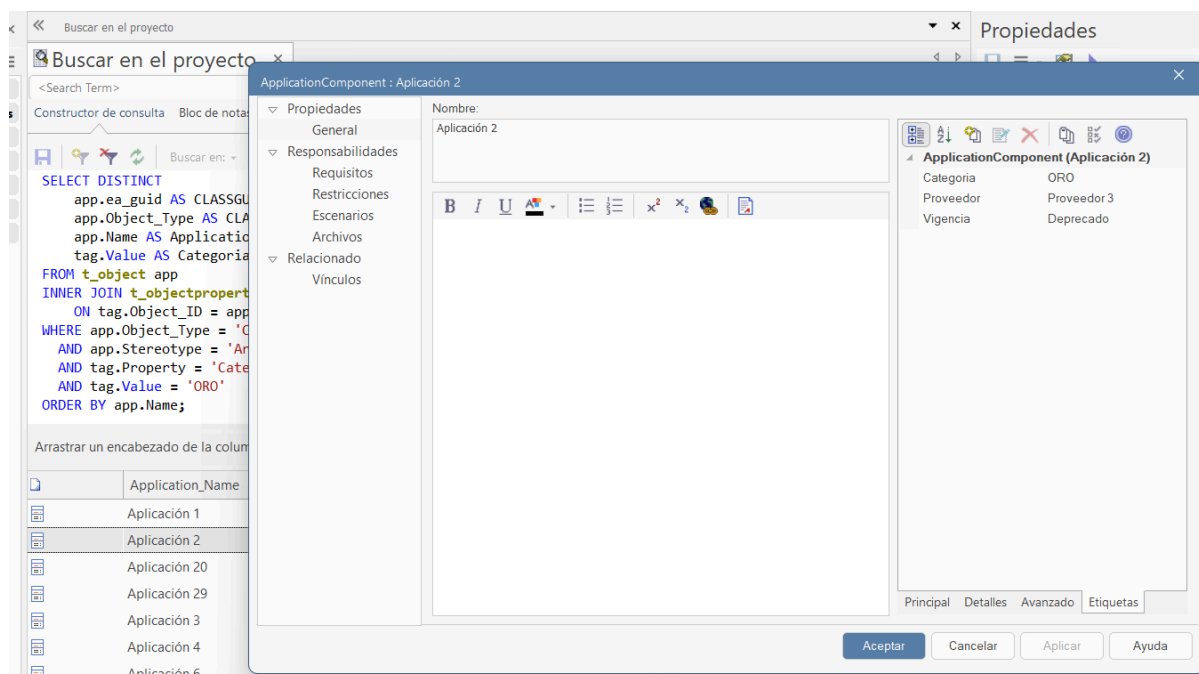
```
app.ea_guid AS CLASSGUID
app.Object_Type AS CLASSTYPE
```

Estas columnas permiten que Enterprise Architect interprete las filas obtenidas como elementos del modelo.

Para comprobar este comportamiento se seleccionó una aplicación directamente desde los resultados de la búsqueda y se realizó la navegación al elemento.

Enterprise Architect abrió correctamente la ventana de propiedades de la aplicación seleccionada —en la evidencia, **Aplicación 2**—, demostrando que el resultado SQL mantiene trazabilidad hacia el elemento real almacenado en el modelo.

Resultado de navegación: PASS.



Resultado de Q1

Las dos consultas fueron ejecutadas correctamente en Enterprise Architect.

Validación	Resultado
Q1-C ejecuta correctamente en EA	PASS
Conteo obtenido	8
Q1-L ejecuta correctamente en EA	PASS
Cantidad de aplicaciones listadas	8
Aplicaciones obtenidas	Coinciden con el resultado esperado
Categoría en resultados	ORO
Elementos no aplicación	Ninguno
Navegación mediante CLASSGUID / CLASSTYPE	PASS

Por lo tanto, las consultas desarrolladas responden correctamente al primer interrogante del desafío y permiten tanto obtener el indicador agregado como navegar hacia los elementos responsables del resultado.

Q2 – Aplicaciones por estado de Vigencia

La segunda consulta responde al interrogante:

¿Cuántas aplicaciones se encuentran en cada estado de vigencia: Vigente y Deprecado?

A diferencia de Q1, esta pregunta requiere solamente un resultado agregado, por lo que decidí implementar una sola consulta denominada Q2-G — Vigencia — Grouped.

La consulta relaciona `t_object` con `t_objectproperties` mediante `Object_ID`, limita el universo a elementos `Class` con estereotipo `ArchiMate_ApplicationComponent` y utiliza exclusivamente el tagged value `Vigencia`. No se utiliza el campo `Status` del elemento, debido a que dicho campo no representa el concepto de vigencia solicitado por el ejercicio.

Para evitar que una eventual duplicación de filas del tagged value aumente incorrectamente los resultados, el agregado utiliza `COUNT(DISTINCT app.Object_ID)`.

La consulta no incorpora `CLASSGUID` ni `CLASSTYPE`, ya que cada fila representa un grupo de aplicaciones y no un elemento individual navegable.

El diagnóstico previo read-only sobre el repositorio obtuvo `Vigente = 16` y `Deprecado = 14`, sin valores `NULL`, vacíos o `N/A`, y sin aplicaciones con valores de Vigencia conflictivos.

La validación funcional definitiva se realizó posteriormente en Enterprise Architect mediante la búsqueda Q2-G — Vigencia — Grouped. Los valores obtenidos fueron comparados con los tagged values Vigencia visibles en elementos del modelo.

Constructor de consulta Bloc de notas SQL

Buscar en: Acción de sub-filtro

```

SELECT
    tag.Value AS Vigencia,
    COUNT(DISTINCT app.Object_ID) AS Application_Count
FROM t_object app
INNER JOIN t_objectproperties tag
    ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
    AND app.Stereotype = 'ArchiMate_ApplicationComponent'
    AND tag.Property = 'Vigencia'
    AND tag.Value IN ('Vigente', 'Deprecado')
GROUP BY tag.Value
ORDER BY tag.Value;

```

Arrastrar un encabezado de la columna aquí para agrupar por esa columna.

Vigencia	Application_Count
Deprecado	14
Vigente	16

Q3 – Impacto por baja de Base de Datos 28

La tercer pregunta planteada por el desafío es:

¿Qué aplicaciones se verían afectadas si se da de baja la Base de Datos 28?

Para responder, fue necesario analizar las relaciones almacenadas en el repositorio de Enterprise Architect e identificar aquellas dependencias cuyo **origen es una aplicación** y cuyo **destino es Base de Datos 28**.

A diferencia de Q1 y Q2, esta consulta no trabaja con tagged values, sino principalmente con las relaciones almacenadas en t_connector.

Las tablas utilizadas son:

- **t_object** para representar la aplicación origen.
- **t_connector** para obtener las relaciones.
- **t_object** nuevamente para identificar el elemento destino.

La relación se interpreta mediante:

Start_Object_ID = elemento origen

End_Object_ID = elemento destino

y se consideran únicamente conectores de tipo:

Dependency

Identificación de Base de Datos 28

El destino no se identifica mediante un identificador interno fijo. Aunque durante el análisis diagnóstico, se observó que Base de Datos 28 posee actualmente Object_ID = 102, dicho valor se utilizó únicamente como referencia de validación.

Las consultas finales identifican el elemento utilizando propiedades semánticas:

Name = Base de Datos 28

Object_Type = Class

Stereotype = ArchiMate_DataObject

De esta manera, la consulta no depende de una clave interna específica del repositorio y resulta más reutilizable. Antes de ejecutar las consultas finales se verificó mediante diagnóstico read-only que este criterio identifica exactamente un único elemento.

División de Q3 en dos consultas

Al igual que en Q1, se decidió separar la respuesta en dos consultas:

- **Q3-C — DB28 Impact — Count:** obtiene la cantidad total de aplicaciones afectadas.
- **Q3-L — DB28 Impact — List:** devuelve las aplicaciones concretas que dependen de la base de datos.

La separación permite mantener una consulta agregada simple para el indicador total y una segunda consulta orientada a trazabilidad.

Q3-L incorpora además:

```
src.ea_guid AS CLASSGUID
src.Object_Type AS CLASSTYPE
```

lo que permite que Enterprise Architect reconozca cada fila como un elemento real del modelo y habilite la navegación directa hacia la aplicación.

Prueba funcional Q3-C — Cantidad de aplicaciones afectadas

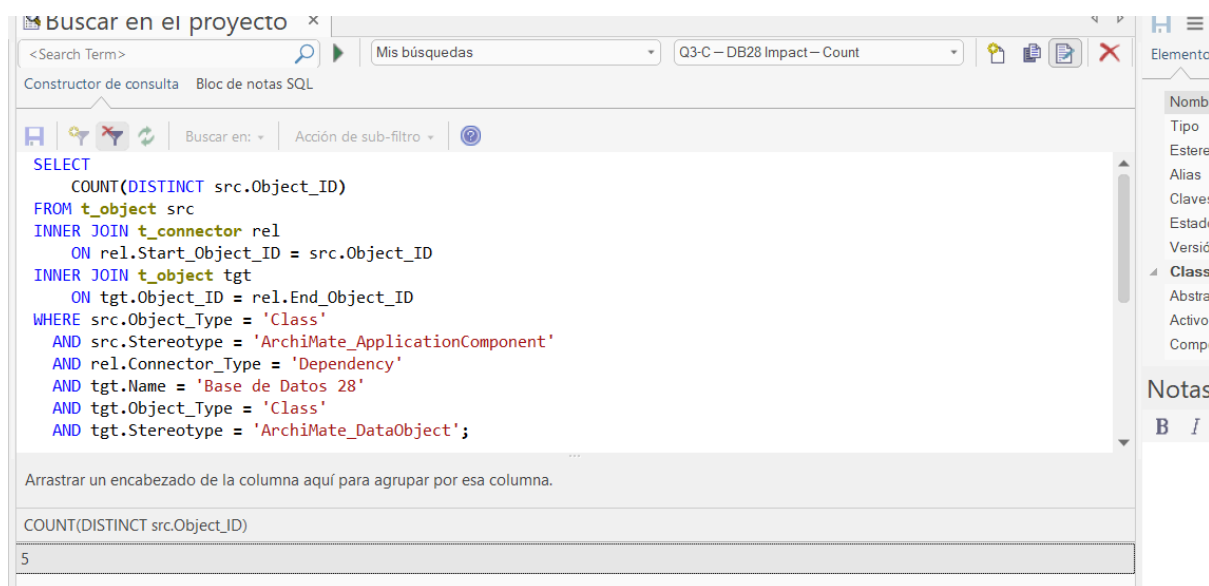
Se creó en Enterprise Architect la búsqueda:

Q3-C — DB28 Impact — Count

La consulta ejecutada identifica relaciones **Dependency** cuyo origen cumple el criterio de aplicación y cuyo destino corresponde semánticamente a **Base de Datos 28**. El resultado obtenido en Enterprise Architect fue:

5

El resultado coincide exactamente con el valor obtenido previamente durante la validación diagnóstica read-only del repositorio.



Prueba funcional Q3-L – Listado de aplicaciones afectadas

Se creó una segunda búsqueda:

Q3-L — DB28 Impact — List

La consulta devolvió exactamente cinco aplicaciones:

Aplicación 12, Aplicación 22, Aplicación 25, Aplicación 27, Aplicación 5

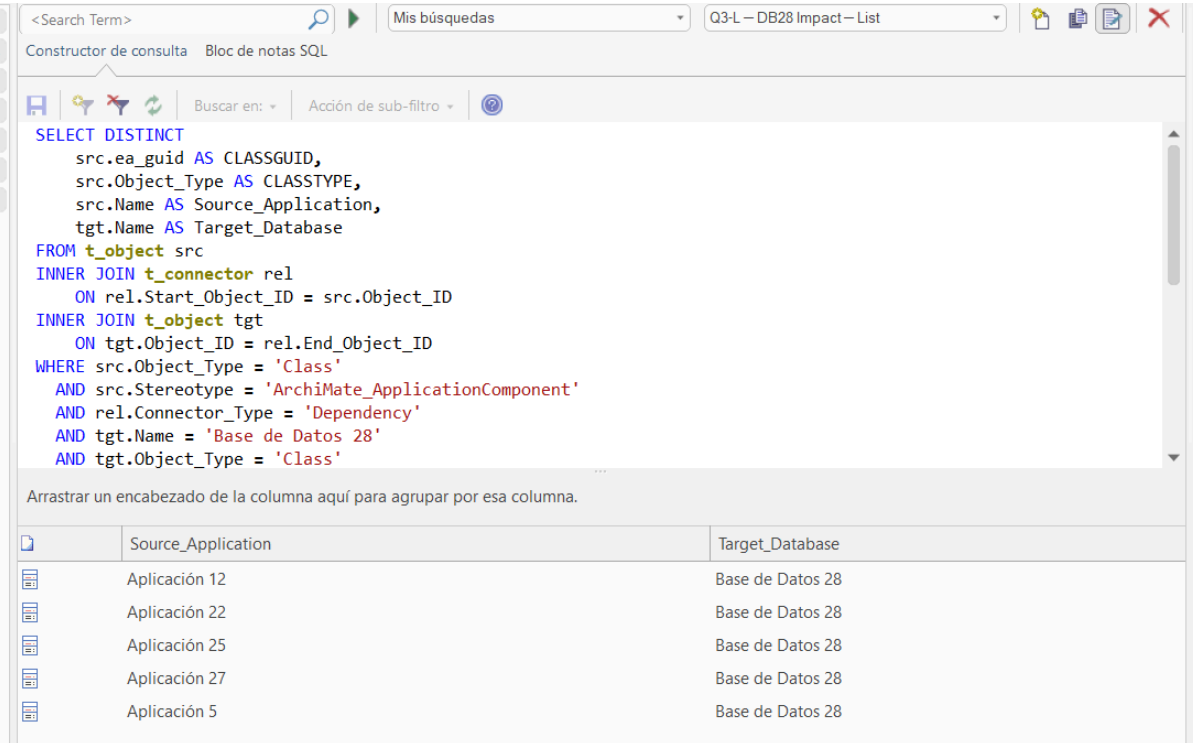
El orden observado responde al orden alfabético aplicado sobre src.Name; por lo tanto, no afecta la comparación con el conjunto esperado. El conjunto obtenido corresponde exactamente a:

Aplicación 5, Aplicación 12, Aplicación 22, Aplicación 25, Aplicación 27

Además, para cada fila el resultado indicó:

Target_Database = Base de Datos 28

No aparecieron elementos de infraestructura ni otros tipos de elementos.



The screenshot shows a SQL query editor with a search bar at the top containing "Q3-L — DB28 Impact — List". Below the search bar, there's a section for the SQL query. The query is as follows:

```
SELECT DISTINCT
  src.ea_guid AS CLASSGUID,
  src.Object_Type AS CLASSTYPE,
  src.Name AS Source_Application,
  tgt.Name AS Target_Database
FROM t_object src
INNER JOIN t_connector rel
  ON rel.Start_Object_ID = src.Object_ID
INNER JOIN t_object tgt
  ON tgt.Object_ID = rel.End_Object_ID
WHERE src.Object_Type = 'Class'
AND src.Stereotype = 'ArchiMate_ApplicationComponent'
AND rel.Connector_Type = 'Dependency'
AND tgt.Name = 'Base de Datos 28'
AND tgt.Object_Type = 'Class'
```

Below the query, there's a table with the results. The table has two columns: "Source_Application" and "Target_Database". The results are as follows:

Source_Application	Target_Database
Aplicación 12	Base de Datos 28
Aplicación 22	Base de Datos 28
Aplicación 25	Base de Datos 28
Aplicación 27	Base de Datos 28
Aplicación 5	Base de Datos 28

Exclusión de elementos no aplicación

Durante el análisis diagnóstico del repositorio se detectaron también relaciones hacia Base de Datos 28 provenientes de:

Servidor 2
Servidor 5
Servidor 19

Estos elementos corresponden a nodos de infraestructura y no a aplicaciones. Por ese motivo, las consultas Q3 restringen explícitamente el origen mediante:

Object_Type = Class
Stereotype = ArchiMate_ApplicationComponent

La ejecución funcional en EA confirmó que ninguno de estos servidores aparece en Q3-L.

Esta comprobación demuestra que la consulta responde específicamente a la pregunta del desafío sobre **aplicaciones afectadas**, y no a todos los elementos relacionados con la base de datos.

Prueba de navegación desde Q3-L

Para comprobar la trazabilidad de los resultados, se realizó doble clic sobre una de las filas retornadas por Q3-L.

En la prueba se seleccionó: Aplicación 22

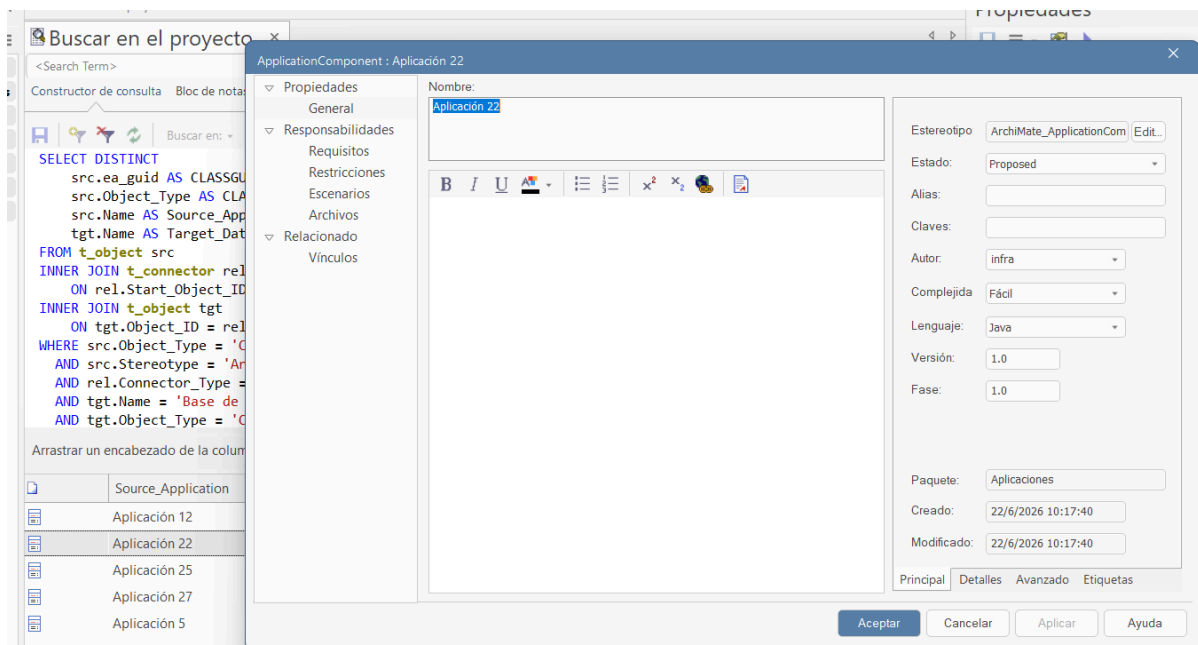
Enterprise Architect abrió correctamente la ventana de propiedades correspondiente a **Aplicación 22**.

Esto confirma que las columnas:

CLASSGUID
CLASSTYPE

permiten relacionar el resultado SQL con el elemento real almacenado en el modelo.

En la ventana de propiedades también fue posible visualizar los tagged values propios de la aplicación, confirmando que se había navegado efectivamente al elemento correspondiente y no a una representación textual sin vínculo con el modelo.



Verificación de la relación con Base de Datos 28

Desde las propiedades de **Aplicación 22** se accedió a:

Relacionado → Vínculos

En esta sección Enterprise Architect mostró entre sus relaciones:

Elemento: Base de Datos 28

Esteriotipo del elemento: ArchiMate_DataObject

Tipo: Clase

Conexión: Dependencia

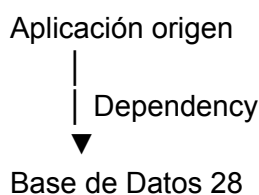
Esta evidencia confirma visualmente que la aplicación seleccionada posee una relación de tipo **Dependencia** con **Base de Datos 28**.

La orientación utilizada por la consulta se establece en SQL mediante:

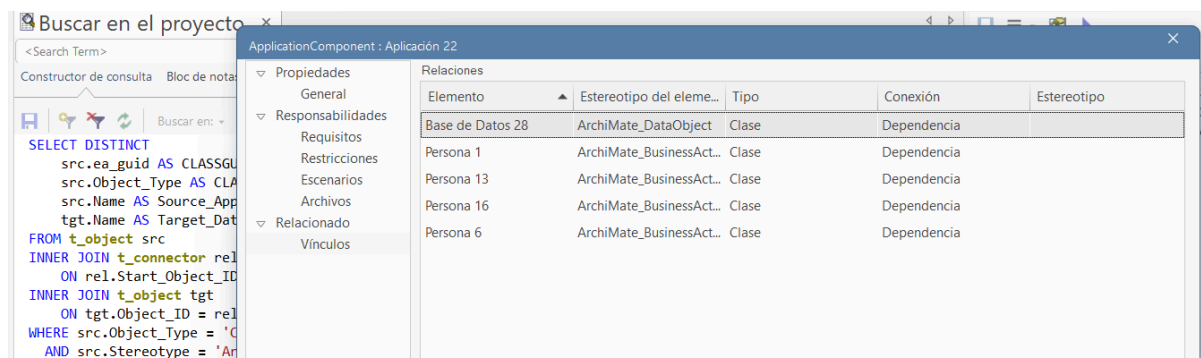
rel.Start_Object_ID = src.Object_ID

rel.End_Object_ID = tgt.Object_ID

es decir:



La interfaz de EA confirma la existencia y el tipo de la relación hacia el elemento destino, mientras que la estructura del repositorio y los joins utilizados determinan la orientación source → target utilizada por la consulta.



Resultado final de Q3

Validación	Resultado
Q3-C ejecuta correctamente en EA	PASS
Cantidad de aplicaciones afectadas	5
Q3-L ejecuta correctamente en EA	PASS
Aplicaciones obtenidas	5, 12, 22, 25 y 27
Destino de las relaciones	Base de Datos 28
Tipo de relación	Dependency / Dependencia
Servidor 2, 5 y 19 excluidos	PASS
Navegación desde Q3-L	PASS
Relación visible en el modelo	PASS
Uso de Object_ID = 102 como filtro	No
Uso de Direction como filtro	No

Las pruebas realizadas confirman que Q3 responde correctamente al interrogante de análisis de impacto planteado por el desafío. La solución obtiene tanto el indicador agregado como el detalle de las aplicaciones afectadas y mantiene trazabilidad hacia los elementos y relaciones existentes en Enterprise Architect.